

Tests de AC - Todos los temas

Los temas recopilados y "verificados por expertos de computadores xd":

Tema 1: <https://forms.gle/p2DEHktHKqfxJ6mh7>

Tema 2: <https://forms.gle/KcakR4QFeDdhGM8YA>

Tema 3: <https://forms.gle/6sKwjeN7DkNsnrGn7>

Tema 4: <https://forms.gle/drkJL85soWzhX2WS6>

Enlace a este cuestionario: <https://forms.gle/T4fDHmRiu1TxGQFk7>

Las practicas (bp) están en otro cuestionario para que éste no sea demasiado largo:

<https://forms.gle/ExtHLXdkpLg9pY6>

Créditos a @paco por hacer los tests de las bp que yo no daba a basto

Hay montado un drive de AC con toda clase de material, te puedes meter desde aquí:

https://drive.google.com/drive/folders/1snGi9YJwufhnecTU_vSkdyclJgE5PjvD?usp=sharing

Aún así aviso que hay más de 100 preguntas, se tarda mucho en hacerlo entero (+1h estimada).

1. [Navegación del cuestionario:]

Marca solo un óvalo.

- ☐ Volver al principio para elegir cuestionario por Tema
- ☐ Tema 1 *Salta a la pregunta 2*
- ☐ Tema 2 *Salta a la pregunta 24*
- ☐ Tema 3 *Salta a la pregunta 63*
- ☐ Tema 4 *Salta a la pregunta 105*
- ☐ Terminar Cuestionario

Tema
1

Son 21 preguntas, al final podrás elegir si enviar el formulario o seguir con otro tema

2. Selecciona los criterios de clasificación del paralelismo implícito:

1 punto

Selecciona todos los que correspondan.

- ☐ Paralelismo de tareas - paralelismo de datos
- ☐ Granularidad
- ☐ Niveles de P.I en un código que resuelve la aplicación
- ☐ Cláusulas
- ☐ Tiempo

3. Es de grano fino

1 punto

Marca solo un óvalo.

- ☐ Programas: Programa1, programa2, ...
- ☐ Funciones: Func1(), Func2()...
- ☐ Bucle(bloques): for, while...
- ☐ Operaciones: +, *, +, -

4. Es de grano grueso

1 punto

Marca solo un óvalo.

- ☐ Programas: Programa1, programa2, ...
- ☐ Funciones: Func1(), Func2()...
- ☐ Bucle(bloques): for, while...
- ☐ Operaciones: +, *, +, -

5. Que dependencia tiene el siguiente fragmento:

1 punto

```
...  
a=b+c  
... //código que no usa a  
d=a+c  
...
```

Marca solo un óvalo.

- ☐ RAW (Read After Write)
- ☐ WAW (Write After Write)
- ☐ WAR (Write After Read)
- ☐ RAR (Read After Read)

6. Que dependencia tiene el siguiente fragmento:

1 punto

```
...  
a=b+c  
... //se lee a  
a=d+e  
... //se lee a
```

Marca solo un óvalo.

- ☐ RAW
- ☐ WAW
- ☐ WAR
- ☐ RAR

7. Que dependencia tiene el siguiente fragmento:

1 punto

```
...  
b=a+1  
...  
a=d+e  
... //se lee a
```

Marca solo un óvalo.

- ☐ RAW
- ☐ WAW
- ☐ WAR
- ☐ RAR

8. Es una característica del Paralelismo de Tareas (task parallelism o TLP - Task Level Par.) 1 punto

Selecciona todos los que correspondan.

- ☐ Se encuentra extrayendo la estructura lógica de funciones de una aplicación.
- ☐ Está relacionado con el paralelismo a nivel de función
- ☐ Se encuentra implícito en las operaciones con estructuras de datos (vectores y matrices)
- ☐ Está relacionado con el paralelismo a nivel de bucle

9. Es una característica del Paralelismo de datos (data parallelism o DLP - Data Level Par.) 1 punto

Selecciona todos los que correspondan.

- ☐ Se encuentra extrayendo la estructura lógica de funciones de una aplicación.
- ☐ Está relacionado con el paralelismo a nivel de función
- ☐ Se encuentra implícito en las operaciones con estructuras de datos (vectores y matrices)
- ☐ Está relacionado con el paralelismo a nivel de bucle

10. La computación paralela: 1 punto

Marca solo un óvalo.

- ☐ Estudia los aspectos hardware y software relacionados con el desarrollo y ejecución de aplicaciones en un sistema de cómputo compuesto por múltiples cores/procesadores/computadores que es visto externamente como una unidad autónoma (multicores, multiprocesadores, multicomputadores, cluster)
- ☐ Estudia los aspectos hardware y software relacionados con el desarrollo y ejecución de aplicaciones en un sistema distribuido; es decir, en una colección de recursos autónomos (PC, servidores -de datos, software, ...-, supercomputadores ...) situados en distintas localizaciones físicas

11. Pertenecen a la clasificación de flynn de arquitecturas (Flujo instr. / flujo de datos): 1 punto

Marca solo un óvalo.

- ☐ SISD, MIMD, MISD, SIMD,
☐ SISD, SMID, MDIM, MISD
☐ SMID, MDIM, MIMD, DISM
☐ UIMD, API, WSP, PC

12. Corresponde a una Arquitectura SISD 1 punto

Marca solo un óvalo.

- ☐ Computadores uni-procesador
☐ Aprovecha el paralismo de datos
☐ Aprovecha paralelismo funcional
☐ No existe físicamente

13. Corresponde a una Arquitectura SIMD 1 punto

Marca solo un óvalo.

- ☐ Computadores uni-procesador
☐ Aprovecha el paralismo de datos
☐ Aprovecha paralelismo funcional
☐ No existe físicamente

14. Corresponde a una Arquitectura MISD 1 punto

Selecciona todos los que correspondan.

- ☐ Computadores uni-procesador
☐ Aprovecha el paralismo de datos
☐ Aprovecha paralelismo funcional
☐ No existe físicamente

15. Corresponde a una Arquitectura MIMD

1 punto

Selecciona todos los que correspondan.

- ☐ Computadores uni-procesador
- ☐ Aprovecha el paralismo de datos
- ☐ Aprovecha paralelismo funcional
- ☐ No existe físicamente

16. Diferencias entre multiprocesador (abrev: MP) y multicomputador (abrev: MC):

1 punto

Selecciona todos los que correspondan.

- ☐ En MP los procesadores comparten el mismo espacio y en MC cada procesador tiene su espacio de direcciones
- ☐ En MP el programador no necesita saber donde se almacenan los datos pero en MC sí
- ☐ En MP los datos están centralizados en memoria secundaria y en MC no.
- ☐ En MP se usa arquitectura MISD y en MC se usa arquitectura SISD

17. Propiedades del SMP (Symmetric Multi-Processor) frente a un Multicomputador:

1 punto

Selecciona todos los que correspondan.

- ☐ Mayor latencia - Poco escalable
- ☐ Menor latencia – Más escalable
- ☐ Comunicación implícita mediante variables compartidas. Datos no duplicados en memoria principal
- ☐ Comunicación explícita mediante software para paso de mensajes (send/receive). Datos duplicados en memoria principal, copia datos

18. ¿Como se incrementa la escalabilidad de un multiprocesador?

1 punto

Selecciona todos los que correspondan.

- ☐ Aumentando la cache del procesador
- ☐ Usando redes de menor latencia y mayor ancho de banda que un bus
- ☐ Distribuir físicamente los módulos de memoria entre los procesadores
- ☐ Añadiendo más buses de dirección
- ☐ Aumentando el número de dispositivos de Entrada/Salida

19. NORMA es un tipo de Multi-computador.

1 punto

Marca solo un óvalo.

- ☐ Verdadero
- ☐ Falso

20. Son Multi-procesadores:

1 punto

Selecciona todos los que correspondan.

- ☐ NUMA
- ☐ UMA
- ☐ COMA
- ☐ BEBA
- ☐ CC-NUMA
- ☐ NUMA-HEY
- ☐ SMP

21. ¿Cómo se calcula el tiempo de cpu?

1 punto

Marca solo un óvalo.

- ☐ $T_{cpu} = NI \times CPI \times T_{ciclo}$
- ☐ $T_{cpu} = NI \times MIPS \times T_{instruccion}$
- ☐ $T_{cpu} = NI \times PCI \times T_{ciclo}$
- ☐ $T_{cpu} = NI \times CPI \times Frecuencia$

22. Si un programa pasa un 25% de su tiempo de ejecución en una máquina realizando instrucciones de coma flotante, y se mejora la máquina haciendo que estas instrucciones se ejecuten en la mitad de tiempo, entonces calcula su mejora de velocidad S: 1 punto

Marca solo un óvalo.

- ☐ $p=2; f=0,75; y S=2/(1+0.75)=1.14$
- ☐ $p=0.5; f=0,75; y S=0.5/(1+0.75)= 0.28$
- ☐ $p=2; f=0,75; y S=2/(1+0.25)=1.6$

23. [Navegación del cuestionario:]

Marca solo un óvalo.

- ☐ Volver al principio para elegir cuestionario por Tema
- ☐ Tema 1 *Salta a la pregunta 2*
- ☐ Tema 2 *Salta a la pregunta 24*
- ☐ Tema 3 *Salta a la pregunta 63*
- ☐ Tema 4 *Salta a la pregunta 105*
- ☐ Terminar Cuestionario

Tema
2

Son 38 preguntas, al final podrás elegir si enviar el formulario o seguir con otro tema

24. La expresión para la ley de Gustafson es $S=f+p*(1-f)$, donde f es la fracción no paralelizable del tiempo de ejecución paralelo y p es el número de procesadores que intervienen.

1 punto

Marca solo un óvalo.

- ☐ Verdadero
- ☐ Falso

25. La ganancia de velocidad que consiguen p procesadores en un código secuencial que tarda un tiempo T_s en ejecutarse en un procesador, con una fracción no paralela de T_s igual a 0, un grado de paralelismo ilimitado y un tiempo de overhead igual a p^2 es:

1 punto

$$\frac{T_s}{\frac{T_s}{p} + p^2}$$

Marca solo un óvalo.

- ☐ Verdadero
- ☐ Falso

26. La ganancia de velocidad que consiguen p procesadores en un código secuencial que tarda un tiempo T_s en ejecutarse en un procesador, con una fracción no paralela de T_s igual a 0, un grado de paralelismo igual a n y un tiempo de overhead igual a 0 es igual a p para $p < n$ 1 punto

Marca solo un óvalo.

- ☐ Verdadero
- ☐ Falso

27. Un programa paralelo tarda 20 ns. Durante 10 ns solo puede ser ejecutado por un procesador y durante los otros 10 ns intervienen 5 procesadores (todos ellos igual de cargados). La sobrecarga se considera despreciable. El valor de la ganancia de velocidad es: 1 punto

Marca solo un óvalo.

- ☐ 1
- ☐ 2
- ☐ 3
- ☐ 4

28. La ganancia de velocidad que consiguen p procesadores en un código secuencial que tarda un tiempo T_s en ejecutarse en un procesador, con una fracción no paralela de T_s igual a f , un grado de paralelismo ilimitado y un tiempo de overhead igual a 0 es: 1 punto

$$\frac{p}{1 + f(p - 1)}$$

Marca solo un óvalo.

- ☐ Verdadero
- ☐ Falso

29. Un programa paralelo tarda 20 ns. Durante 10 ns solo puede ser ejecutado por un procesador y durante los otros 10 ns intervienen 5 procesadores (todos igual de cargados). El valor de la f de la ley de Gustafson es: 1 punto

Marca solo un óvalo.

- ☐ 0,25
☐ 0,5
☐ 0,75
☐ 1

30. Ordena por nivel de abstracción 3 puntos

Marca solo un óvalo por fila.

	Abstracción más alta	Abstracción media	Abstracción más baja
Compiladores paralelos	☐	☐	☐
Lenguajes paralelos y API funciones + Directivas	☐	☐	☐
API funciones	☐	☐	☐

31. Selecciona las comunicaciones colectivas de procesadores uno-a-todos: 1 punto

Selecciona todos los que correspondan.

- ☐ Difusión (broadcast)
☐ Dispersión (scatter)
☐ Reducción
☐ Acumulación (gather)
☐ Recorrido (Scan)

32. Selecciona las comunicaciones colectivas de procesadores todos-a-uno: 1 punto

Selecciona todos los que correspondan.

- ☐ Difusión (broadcast)
- ☐ Dispersión (scatter)
- ☐ Reducción
- ☐ Acumulación (gather)
- ☐ Recorrido (Scan)

33. Selecciona las comunicaciones colectivas de procesadores multiple uno-a-uno: 1 punto

Selecciona todos los que correspondan.

- ☐ Rotación
- ☐ Permutación
- ☐ Reducción
- ☐ Acumulación (gather)
- ☐ Recorrido (Scan)

34. Selecciona las comunicaciones colectivas de procesadores todos-a-todos: 1 punto

Selecciona todos los que correspondan.

- ☐ Todos Difunden (all-broadcast) o chismorreo (gossiping)
- ☐ Todos Dispersan (all-scatter)
- ☐ Permutación
- ☐ Rotación
- ☐ Recorrido (Scan)

35. Selecciona la/s comunicaciones colectivas de Servicios Compuestos: 1 punto

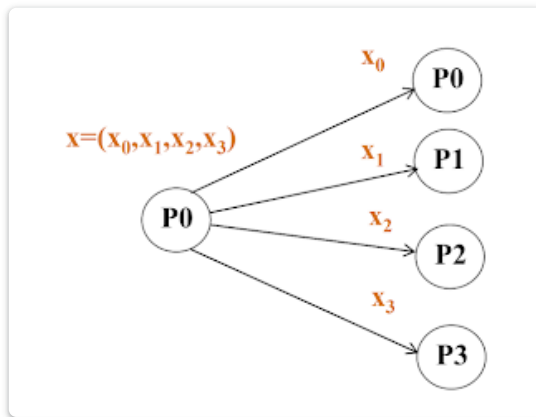
Selecciona todos los que correspondan.

- ☐ Rotación(all-broadcast) o chismorreo (gossiping)
- ☐ Todos Dispersan (all-scatter)
- ☐ Permutación
- ☐ Todos combinan
- ☐ Recorrido (Scan) prefijo paralelo
- ☐ Recorrido sufijo paralelo

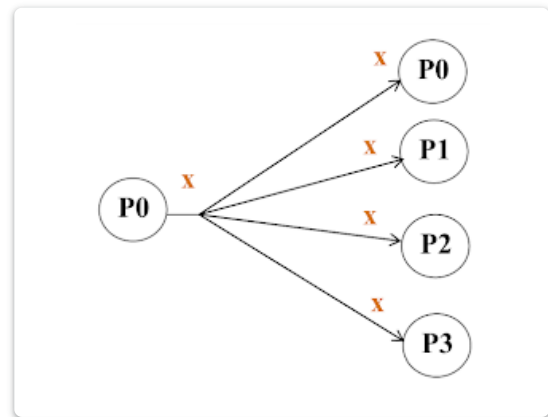
36. Elige la comunicación por: difusión (broadcast)

1 punto

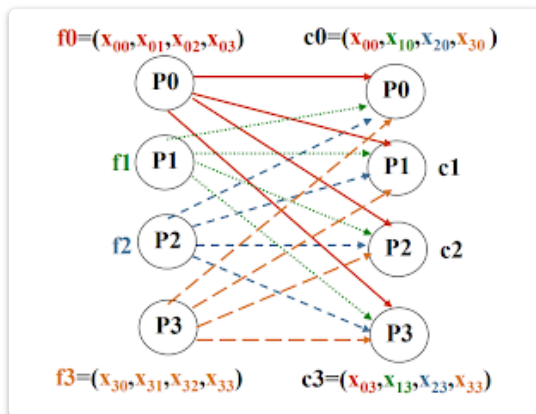
Marca solo un óvalo.



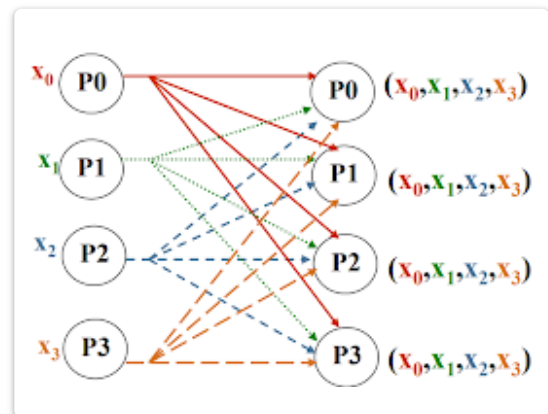
☐ Opción 1



☐ Opción 2



☐ Opción 3

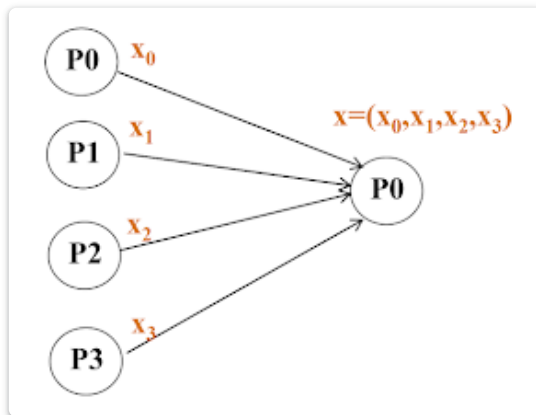


☐ Opción 4

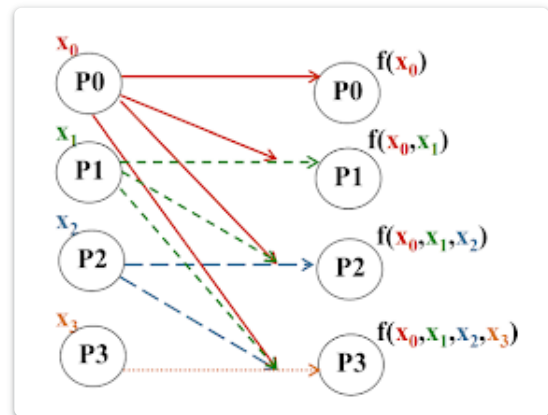
37. Elige la comunicación por: acumulación (gather)

1 punto

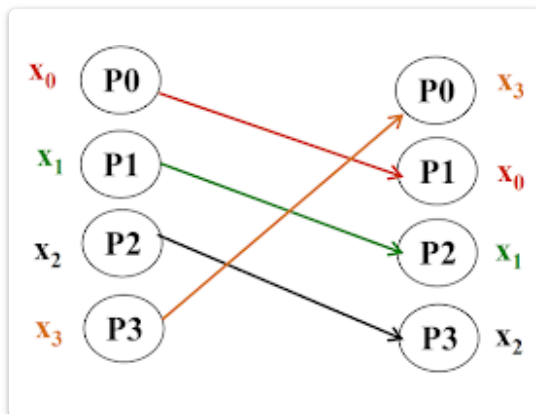
Marca solo un óvalo.



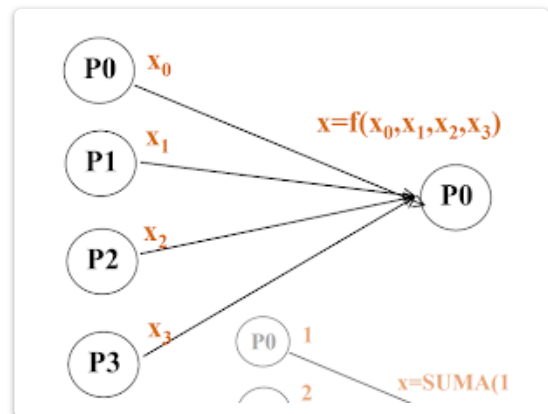
☐ Opción 1



☐ Opción 2



☐ Opción 3

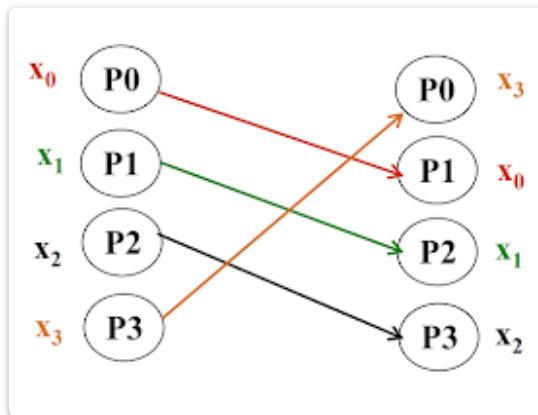


☐ Opción 4

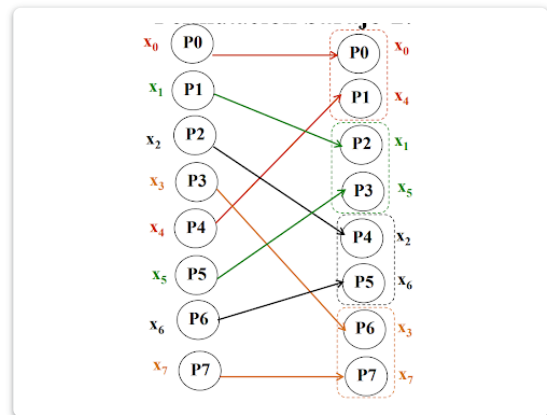
38. Elige la comunicación por: rotación

1 punto

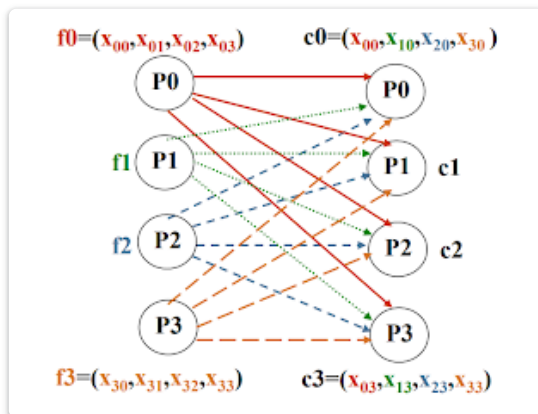
Marca solo un óvalo.



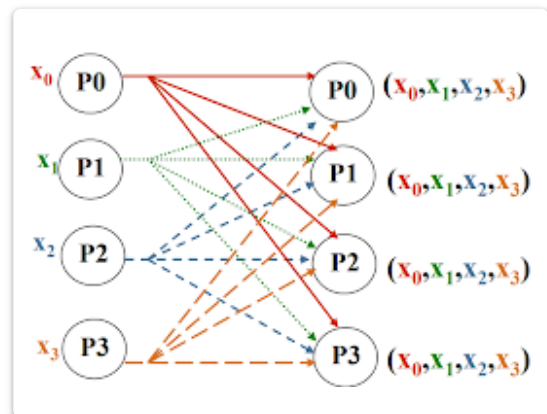
☐ Opción 1



☐ Opción 2



☐ Opción 3

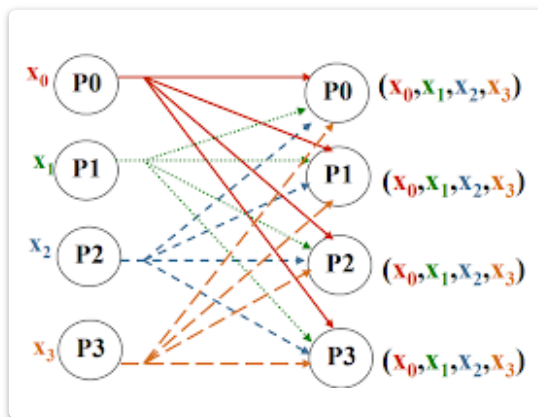


☐ Opción 4

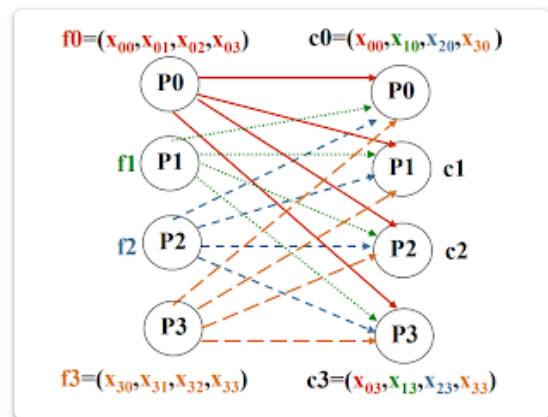
39. Elige la comunicación por: recorrido (scan) prefijo paralelo

1 punto

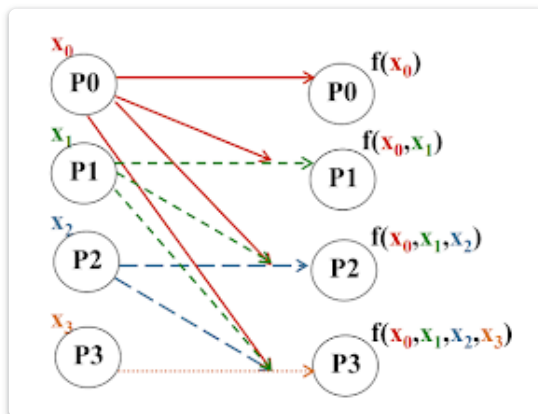
Marca solo un óvalo.



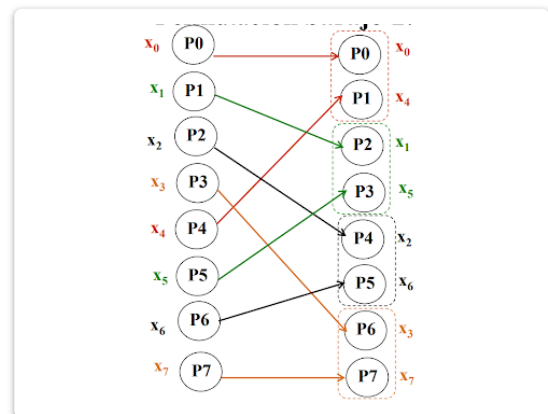
☐ Opción 1



☐ Opción 2



☐ Opción 3



☐ Opción 4

40. ¿Que directivas/cláusulas de OpenMP se corresponden a la comunicación uno-a-todos?

1 punto

Selecciona todos los que correspondan.

- ☐ Cláusula firstprivate
- ☐ Directiva single
- ☐ Directiva threadprivate
- ☐ Cláusula reduction
- ☐ Directiva barrier

41. ¿Que directivas/cláusulas de OpenMP se corresponden a la comunicación todos-a-uno?

1 punto

Selecciona todos los que correspondan.

- ☐ Cláusula firstprivate
- ☐ Directiva single
- ☐ Directiva threadprivate
- ☐ Cláusula reduction
- ☐ Directiva barrier

42. ¿Que directivas/cláusulas de OpenMP se corresponden a la comunicación por Servicios Compuestos?

1 punto

Selecciona todos los que correspondan.

- ☐ Cláusula firstprivate
- ☐ Directiva single
- ☐ Directiva threadprivate
- ☐ Cláusula reduction
- ☐ Directiva barrier

43. ¿Que funciones de MPI se corresponden a la comunicación por uno-a-uno?

1 punto

Marca solo un óvalo.

- ☐ MPI_Send()/MPI_Receive()
- ☐ MPI_Scatter()
- ☐ MPI_Reduce()
- ☐ MPI_Allgather()
- ☐ MPI_Scan()

44. ¿Que funciones de MPI se corresponden a la comunicación por uno-a-todos? 1 punto

Marca solo un óvalo.

- ☐ MPI_Send()/MPI_Receive()
☐ MPI_Scatter()
☐ MPI_Reduce()
☐ MPI_Allgather()
☐ MPI_Scan()

45. ¿Que funciones de MPI se corresponden a la comunicación por todos-a-uno? 1 punto

Marca solo un óvalo.

- ☐ MPI_Send()/MPI_Receive()
☐ MPI_Scatter()
☐ MPI_Reduce()
☐ MPI_Allgather()
☐ MPI_Scan()

46. ¿Que funciones de MPI se corresponden a la comunicación por todos-a-todos? 1 punto

Marca solo un óvalo.

- ☐ MPI_Send()/MPI_Receive()
☐ MPI_Scatter()
☐ MPI_Reduce()
☐ MPI_Allgather()
☐ MPI_Scan()

47. ¿Que funciones de MPI se corresponden a la comunicación por Servicios compuestos?

1 punto

Marca solo un óvalo.

- ☐ MPI_Send()/MPI_Receive()
☐ MPI_Scatter()
☐ MPI_Reduce()
☐ MPI_Allgather()
☐ MPI_Scan()

48. ¿Cuál es el estilo de programación para los multicomputadores?

1 punto

Marca solo un óvalo.

- ☐ Paso de mensajes
☐ Variables compartidas
☐ Paralelismo de datos

49. ¿Cuál es el estilo de programación para los multiprocesadores?

1 punto

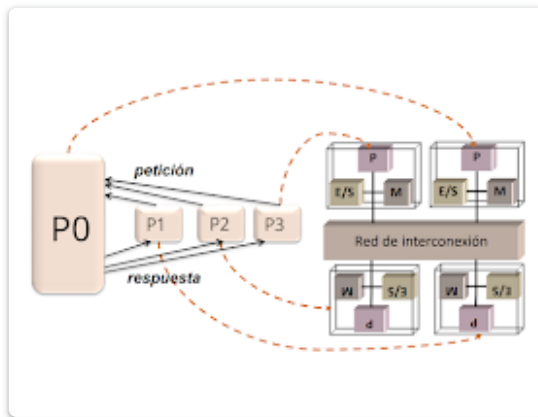
Marca solo un óvalo.

- ☐ Paso de mensajes
☐ Variables compartidas
☐ Paralelismo de datos

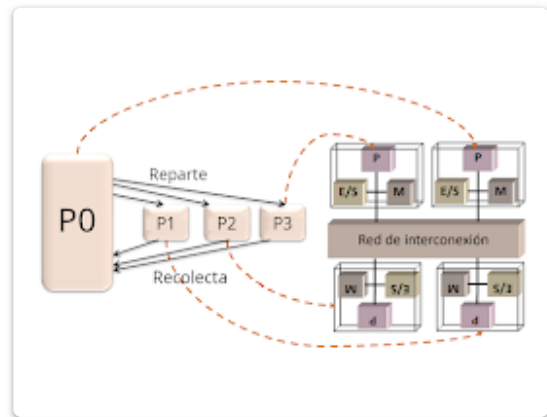
50. Selecciona la estructura de: Master-Slave o granja de tareas

1 punto

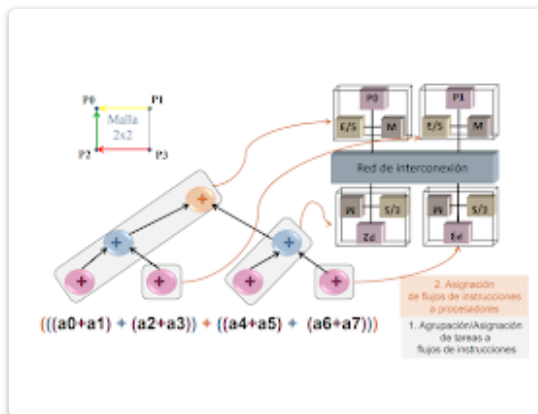
Marca solo un óvalo.



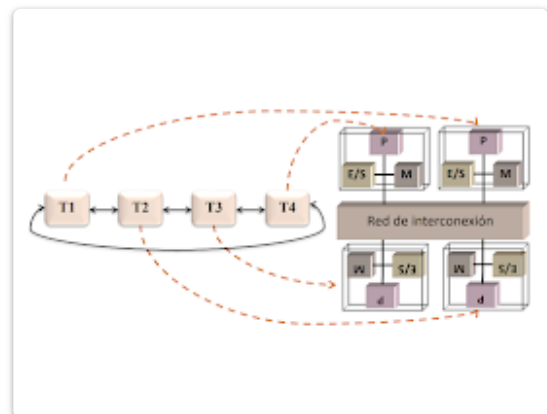
☐ Opción 1



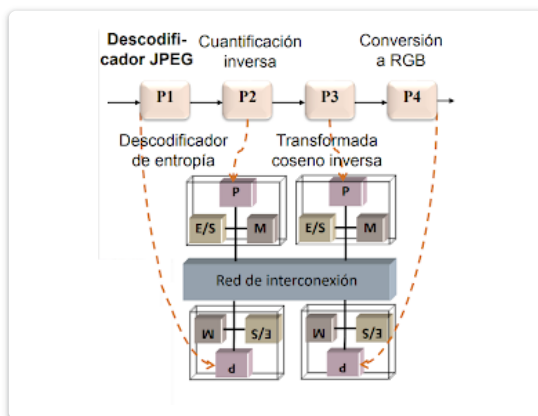
☐ Opción 2



☐ Opción 3



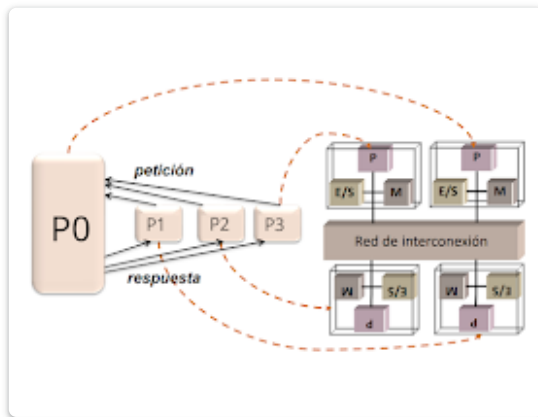
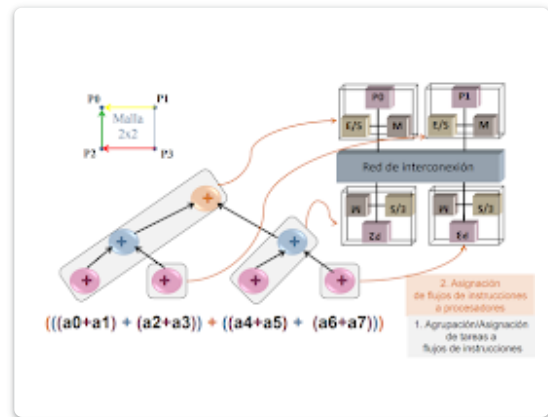
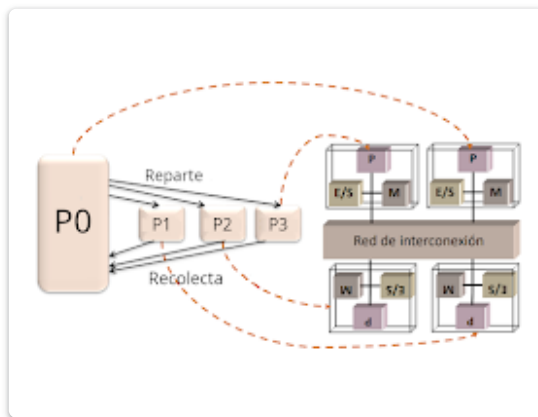
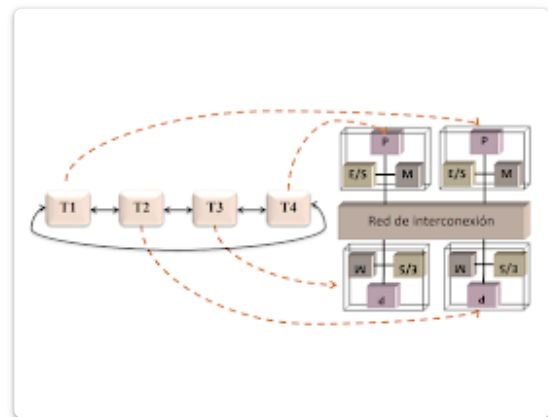
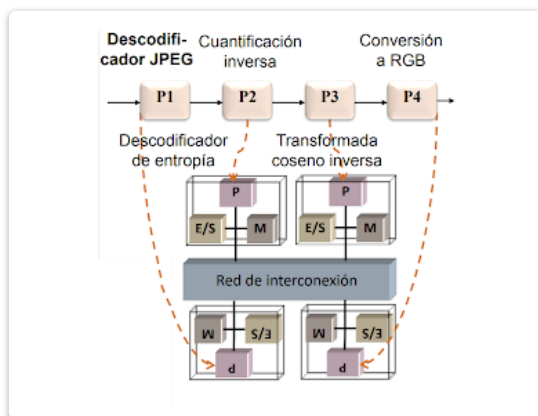
☐ Opción 4



☐ Opción 5

51. Selecciona la estructura de: Cliente-Servidor

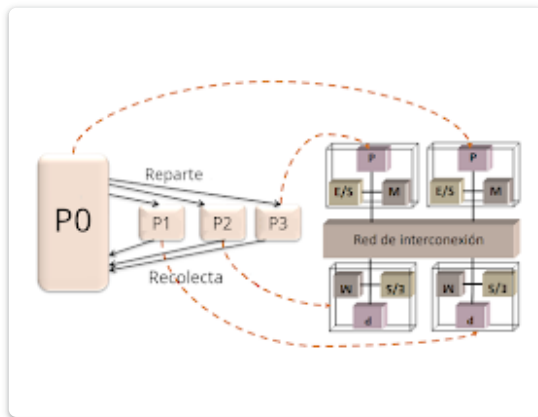
1 punto

Marca solo un óvalo.☐ Opción 1☐ Opción 2☐ Opción 3☐ Opción 4☐ Opción 5

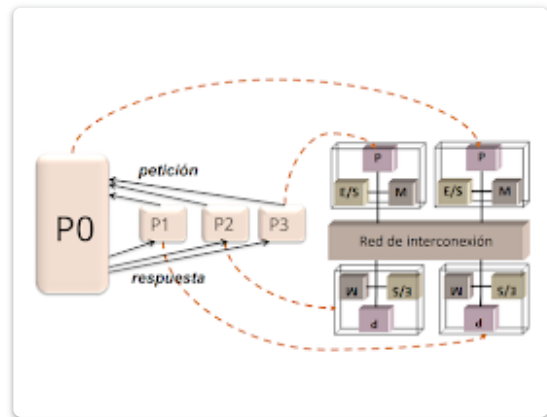
52. Selecciona la estructura de: Descomposición de domino o datos

1 punto

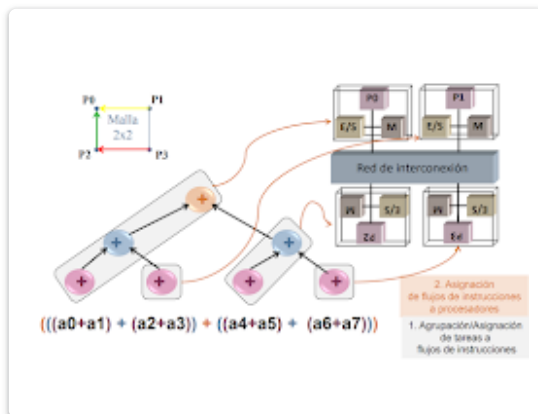
Marca solo un óvalo.



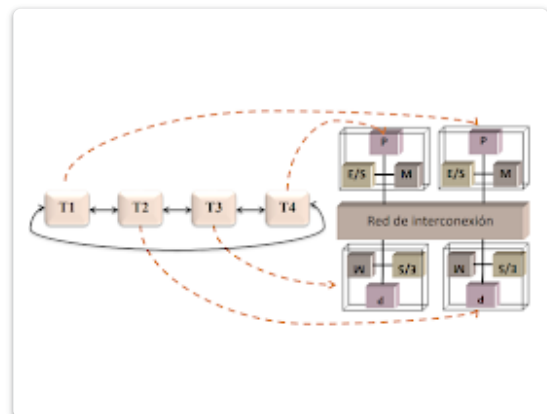
☐ Opción 1



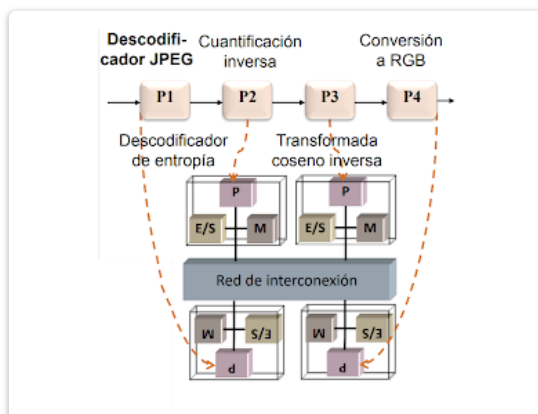
☐ Opción 2



☐ Opción 3



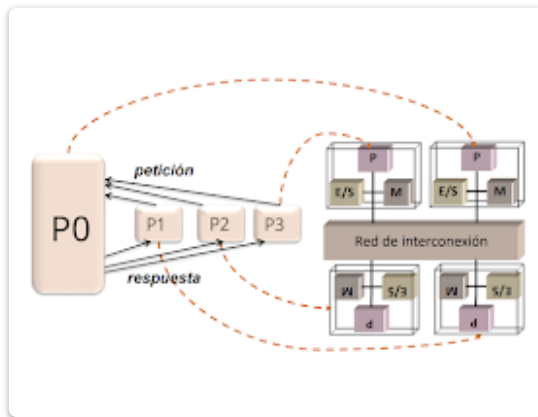
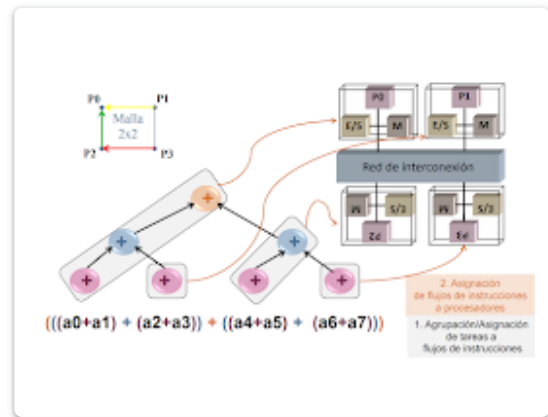
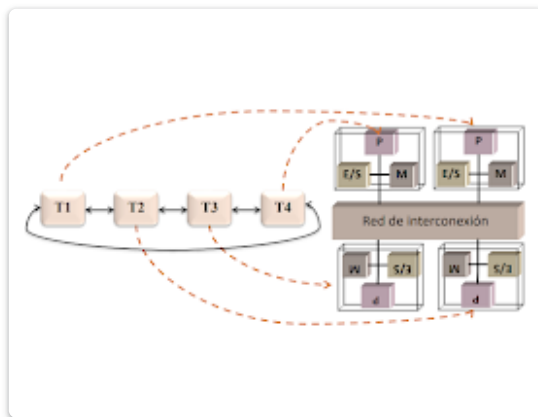
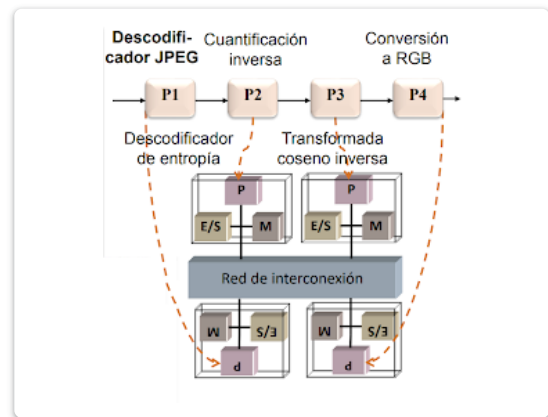
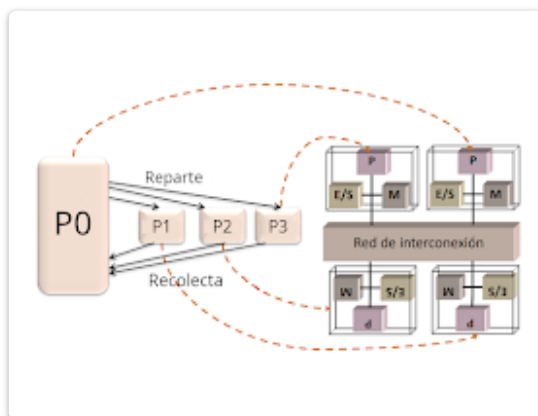
☐ Opción 4



☐ Opción 5

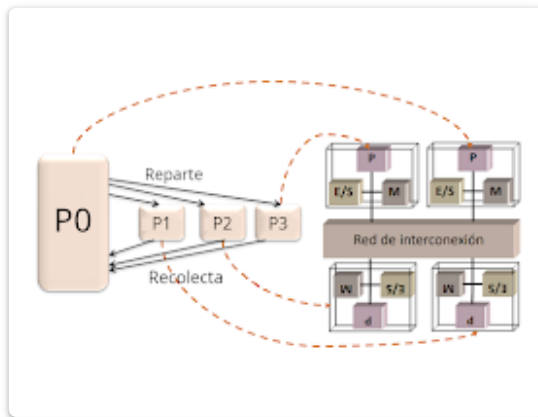
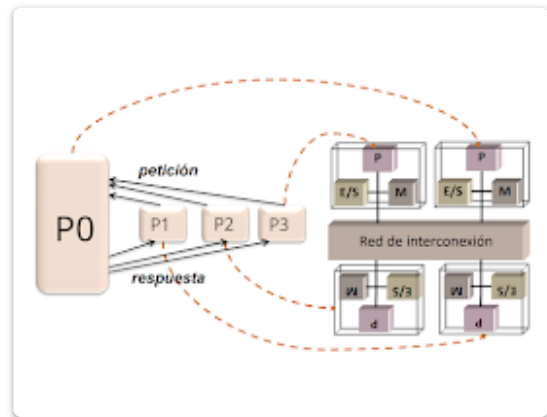
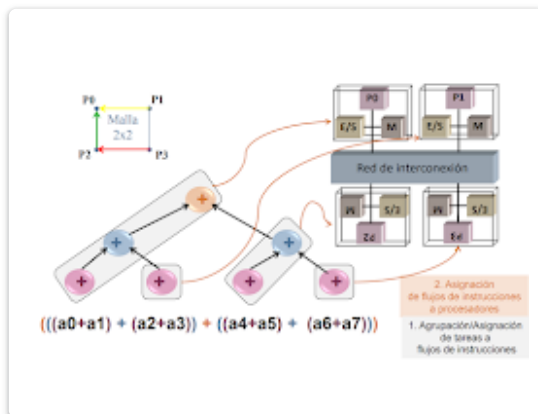
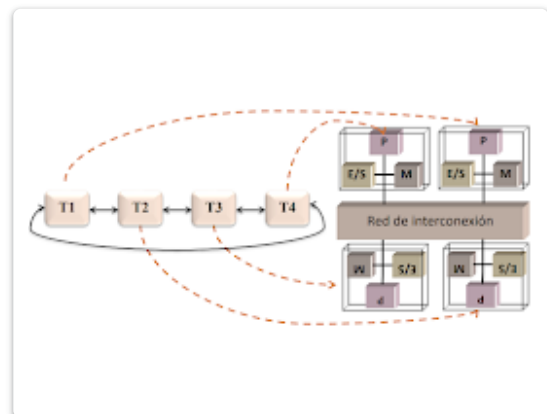
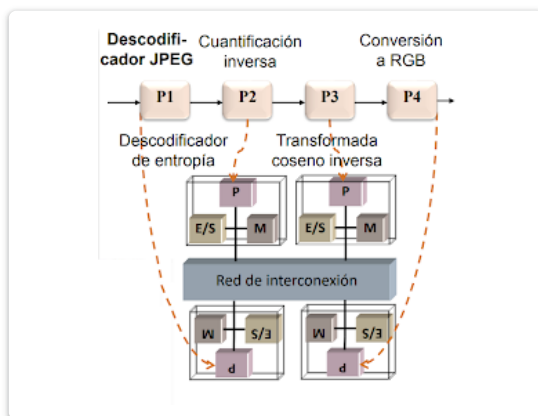
53. Selecciona la estructura de: Estructura Segmentada

1 punto

Marca solo un óvalo.☐ Opción 1☐ Opción 2☐ Opción 3☐ Opción 4☐ Opción 5

54. Selecciona la estructura de: Divide y Vencerás

1 punto

Marca solo un óvalo.☐ Opción 1☐ Opción 2☐ Opción 3☐ Opción 4☐ Opción 5

55. ¿Cual es el orden de los procesos de paralelización?

1 punto

Marca solo un óvalo.

- ☐ 1.Descomponer, 2.Asignar, 3.Redactar, 4.Evaluar
- ☐ 1.Asignar, 2.Descomponer, 3.Evaluar, 4.Redactar
- ☐ 1.Descomponer, 2.Asignar, 3.Evaluar , 4.Redactar
- ☐ 1.Asignar, 2.Descomponer, 3.Redactar, 4. Evaluar

56. Forma parte de la descomposición en tareas independientes

1 punto

Selecciona todos los que correspondan.

- ☐ Análisis de dependencia entre funciones
- ☐ Análisis de dependencia entre iteraciones de bucles
- ☐ Equilibrado de carga
- ☐ Mapeo de Threads

57. ¿De que depende la granularidad de la carga de trabajo asignada a procesos/threads?

1 punto

Selecciona todos los que correspondan.

- ☐ Del número de cores o procesadores o elementos de procesamiento
- ☐ Del tiempo de comunicación/sincronización frente a tiempo de cálculo
- ☐ De la sobrecarga en la asignación de tareas a procesos
- ☐ De la escalabilidad del procesador

58. ¿De que depende el equilibrado de carga?

1 punto

Selecciona todos los que correspondan.

- ☐ De la arquitectura: homogénea frente a heterogenea/uniforme frente a no uniforme
- ☐ De la aplicación/descomposición
- ☐ De la caché del procesador
- ☐ De la comunicación de los procesadores

59. Selecciona la Asignación ESTÁTICA del siguiente bucle:

1 punto

Bucle

```
for (i=0;i<Iter;i++) {
    código para i
}
```

Selecciona todos los que correspondan.

```
for (i=idT;i<Iter;i=i+nT) {
    código para i
}
```

☐ Opción 1

```
for (i=idT *  $\frac{Iter}{nT}$  ; i < (idT+1) *  $\frac{Iter}{nT}$  ; i++) {
    código para i
}
```

☐ Opción 2

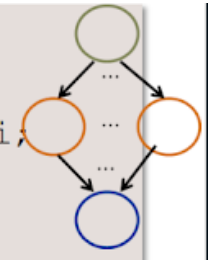
```
lock(k);
n=i; i=i+1;
unlock(k);
while (n<Iter) {
    código para n ;
    lock(k);
    n=i; i=i+1;
    unlock(k);
}
```

☐ Opción 3

60. Elige la redacción correcta:

4 puntos

```
#include <omp.h>
#define NUM_THREADS 4
main(int argc, char **argv) {
    long double ancho,x, sum=0; int intervalos, i;
    intervalos = atoi(argv[1]);
    ancho = 1.0/(double) intervalos;
    omp_set_num_threads(NUM_THREADS);
#pragma omp parallel
{
    #pragma omp for reduction(+:sum) private(x) \
                    schedule(dynamic)
    for (i=0;i< intervalos; i++) {
        x = (i+0.5)*ancho; sum = sum + 4.0/(1.0+x*x);
    }
}
    sum* = ancho;
}
```



Marca solo un óvalo por fila.

	Morado	Naranja	Verde	Amarillo
Crear/Terminar	☐	☐	☐	☐
Localizar	☐	☐	☐	☐
Comunicar/Sincronizar	☐	☐	☐	☐
Agrupar/Asignar	☐	☐	☐	☐

61. ¿En qué situaciones hay sobrecarga (Overhead)?

1 punto

Selecciona todos los que correspondan.

- ☐ En la comunicación/Sincronización
- ☐ Al crear/terminar procesos/threads
- ☐ Cálculos o funciones no presentes en versión secuencial
- ☐ Cálculos o funciones no presentes en versión paralela
- ☐ Al mapear variables

62. [Navegación del cuestionario:]

Marca solo un óvalo.

- ☐ Volver al principio para elegir cuestionario por Tema
- ☐ Tema 1 *Salta a la pregunta 2*
- ☐ Tema 2 *Salta a la pregunta 24*
- ☐ Tema 3 *Salta a la pregunta 63*
- ☐ Tema 4 *Salta a la pregunta 105*
- ☐ Terminar Cuestionario

Tema
3

Son 41 preguntas, al final podrás elegir si enviar el formulario o seguir con otro tema

63. Comparación entre multiprocesador con memoria centralizada (UMA) y multiprocesador con memoria distribuida (NUMA) 1 punto

Marca solo un óvalo.

- ☐ UMA: Mayor latencia - Poco escalable; NUMA: menor latencia - escalable pero requiere de distribución de datos/código
- ☐ NUMA: Mayor latencia - Poco escalable; UMA: menor latencia - escalable pero requiere de distribución de datos/código
- ☐ UMA: Mayor latencia - escalable pero requiere de distribución de datos/código ; NUMA: menor latencia - Poco escalable
- ☐ NUMA: Mayor latencia - escalable pero requiere de distribución de datos/código ; UMA: menor latencia - Poco escalable

64. ¿Que computadores implementan en hardware mantenimiento de coherencia? 1 punto

Selecciona todos los que correspondan.

- ☐ NORMA
- ☐ NUMA
- ☐ CC-NUMA
- ☐ COMA
- ☐ SMP

65. ¿Qué incluye un sistema de memoria en multiprocesadores?

1 punto

Selecciona todos los que correspondan.

- ☐ Caches en todos los nodos
- ☐ Memoria Principal
- ☐ Controladores
- ☐ Buffers
- ☐ Red de interconexión
- ☐ Puertos de entrada/salida
- ☐ Memoria Secundaria

66. ¿Como funciona el método de actualización de: Escritura Inmediata(write-through)?

1 punto

Marca solo un óvalo.

- ☐ Cada vez que un procesador escribe en su cache escribe también en memoria principal
- ☐ Se actualiza la memoria principal escribiendo todo el bloque cuando se desaloja de la caché

67. ¿Como funciona el método de actualización de: Posescritura (write-back)?

1 punto

Marca solo un óvalo.

- ☐ Cada vez que un procesador escribe en su cache escribe también en memoria principal
- ☐ Se actualiza la memoria principal escribiendo todo el bloque cuando se desaloja de la caché

68. Selecciona las alternativas para propagar una escritura en caché:

1 punto

Selecciona todos los que correspondan.

- ☐ Escritura con actualización (write-update)
- ☐ Escritura con invalidación (write-invalidate)
- ☐ Escritura con verificación (write-verify)
- ☐ Escritura con eliminación (write-elimination)

69. ¿Como funciona la Escritura con actualización (write-update) a la hora de propagar una escritura en caché? 1 punto

Marca solo un óvalo.

- ☐ Cada vez que un procesador escribe en una dirección en su caché se escribe en las copias de esa dirección en otras caches
- ☐ Antes que un procesador modifique una dirección en su cache se invalidan las copias del bloque de la dirección en otras cachés

70. ¿Como funciona la Escritura con invalidación (write-invalidate) a la hora de propagar una escritura en caché? 1 punto

Marca solo un óvalo.

- ☐ Cada vez que un procesador escribe en una dirección en su caché se escribe en las copias de esa dirección en otras caches
- ☐ Antes que un procesador modifique una dirección en su cache se invalidan las copias del bloque de la dirección en otras cachés

71. Requisitos del SM para evitar problemas por incoherencia 1 punto

Selecciona todos los que correspondan.

- ☐ Propagar la escritura en una direccion haciendola visible en un tiempo finito a otros procesadores
- ☐ Serializar las escrituras en una dirección: deben verse en el mismo orden por todos los procesadores
- ☐ Propagar la escritura en una dirección usando difusión y para conseguir mayor escalabilidad
- ☐ Serializar las escrituras en una dirección mediante el orden en el que las peticiones llegan a su home (nodo que tiene en MP la dirección)

72. Requisitos del SM para evitar problemas por incoherencia SI LA RED NO ES UN BUS 1 punto

Selecciona todos los que correspondan.

- ☐ Propagar la escritura en una dirección haciéndola visible en un tiempo finito a otros procesadores
- ☐ Serializar las escrituras en una dirección: deben verse en el mismo orden por todos los procesadores
- ☐ Propagar la escritura en una dirección usando difusión y para conseguir mayor escalabilidad
- ☐ Serializar las escrituras en una dirección mediante el orden en el que las peticiones llegan a su home (nodo que tiene en MP la dirección)

73. ¿Para que sirven los protocolos de espionaje (snoopy)? 1 punto

Marca solo un óvalo.

- ☐ Para buses, y en general sistemas con una difusión eficiente (bien porque el número de nodos es pequeño o porque la red implementa difusión))
- ☐ Para redes sin difusión o escalables (multietapa y estáticas)
- ☐ Para redes jerárquicas: jerarquía de buses, jerarquía de redes escalables, redes escalables-buses

74. ¿Para que sirven los protocolos basados en directorios? 1 punto

Marca solo un óvalo.

- ☐ Para buses, y en general sistemas con una difusión eficiente (bien porque el número de nodos es pequeño o porque la red implementa difusión))
- ☐ Para redes sin difusión o escalables (multietapa y estáticas)
- ☐ Para redes jerárquicas: jerarquía de buses, jerarquía de redes escalables, redes escalables-buses

75. ¿Para que sirven los esquemas jerárquicos a la hora de mantener coherencia en el SM? 1 punto

Marca solo un óvalo.

- ☐ Para buses, y en general sistemas con una difusión eficiente (bien porque el número de nodos es pequeño o porque la red implementa difusión))
- ☐ Para redes sin difusión o escalables (multietapa y estáticas)
- ☐ Para redes jerárquicas: jerarquía de buses, jerarquía de redes escalables, redes escalables-buses

76. Selecciona las políticas de actualización de memoria principal implementados en caches: 1 punto

Selecciona todos los que correspondan.

- ☐ Escritura inmediata
- ☐ Posescritura
- ☐ Escritura con invalidación
- ☐ Escritura con actualización

77. Selecciona las políticas de propagación de escritura en protocolos de coherencia de caché: 1 punto

Selecciona todos los que correspondan.

- ☐ Escritura inmediata
- ☐ Posescritura
- ☐ Escritura con invalidación
- ☐ Escritura con actualización

78. Posibles estados del protocolo MSI 1 punto

Marca solo un óvalo.

- ☐ Modificado(M), Compartido(S), Inválido(I)
- ☐ Modificado(M), Compartido(S), Inválido(I), Exclusivo(E)
- ☐ Modificado(M), Servido(S), Inutilizado(I)
- ☐ Memorizado(M), Compartido(S), Insertado(I)

79. Posibles estados del protocolo MESI

1 punto

Marca solo un óvalo.

- ☐ Modificado(M), Compartido(S), Inválido(I)
- ☐ Modificado(M), Compartido(S), Inválido(I), Exclusivo(E)
- ☐ Modificado(M), Servido(S), Inutilizado(I), Expulsado(E)
- ☐ Memorizado(M), Compartido(S), Insertado(I), Excluido(E)

80. Que significa que el estado de una caché es Modificado(M):

1 punto

Marca solo un óvalo.

- ☐ Es la única copia del bloque válida en todo el sistema
- ☐ Está valido en memoria y puede que haya copia válida en otras cachés
- ☐ Se ha invalidado o no está físicamente

81. Que significa que el estado de una caché es Compartido(S):

1 punto

Marca solo un óvalo.

- ☐ Es la única copia del bloque válida en todo el sistema
- ☐ Está valido en memoria y puede que haya copia válida en otras cachés
- ☐ Se ha invalidado o no está físicamente

82. Que significa que el estado de una caché es Exclusivo (E):

1 punto

Marca solo un óvalo.

- ☐ Es la única copia del bloque válida en todo el sistema
- ☐ Está valido en memoria y puede que haya copia válida en otras cachés
- ☐ Se ha invalidado o no está físicamente
- ☐ Es la única copia del bloque válida en caches, la memoria también está actualizada

83. Que significa que el estado de una caché es Inválido(I):

1 punto

Marca solo un óvalo.

- ☐ Es la única copia del bloque válida en todo el sistema
- ☐ Está valido en memoria y puede que haya copia válida en otras cachés
- ☐ Se ha invalidado o no está físicamente

84. En un protocolo MSI, ¿Cuándo se produce una petición de lectura (PtLec) de un bloque?

1 punto

Marca solo un óvalo.

- ☐ Al hacer una lectura con fallo de cache del procesador del nodo (PrLec)
- ☐ Al hacer una escritura del procesador (PrEsc) en bloque compartido o inválido
- ☐ Al hacer un reemplazo del bloque modificado (el procesador del nodo no esperaba respuesta)
- ☐ Al tener un estado modificado el bloque solicitado por una PtLec o PtLecEx recibida

85. En un protocolo MSI, ¿Cuándo se produce una petición de acceso exclusivo (PtLecEx)?

1 punto

Marca solo un óvalo.

- ☐ Al hacer una lectura con fallo de cache del procesador del nodo (PrLec)
- ☐ Al hacer una escritura del procesador (PrEsc) en bloque compartido o inválido
- ☐ Al hacer el reemplazo del bloque modificado (el procesador del nodo no esperaba respuesta)
- ☐ Al tener un estado modificado el bloque solicitado por una PtLec o PtLecEx recibida

86. En un protocolo MSI, ¿Cuándo se produce una petición de posescritura (PtPEsc)? 1 punto

Marca solo un óvalo.

- ☐ Al hacer una lectura con fallo de cache del procesador del nodo (PrLec)
- ☐ Al hacer una escritura del procesador (PrEsc) en bloque compartido o inválido
- ☐ Al hacer el reemplazo del bloque modificado (el procesador del nodo no esperaba respuesta)
- ☐ Al tener un estado modificado el bloque solicitado por una PtLec o PtLecEx recibida

87. En un protocolo MSI, ¿Cuando se produce una respuesta con bloque (RpBloque)? 1 punto

Marca solo un óvalo.

- ☐ Al hacer una lectura con fallo de cache del procesador del nodo (PrLec)
- ☐ Al hacer una escritura del procesador (PrEsc) en bloque compartido o inválido
- ☐ Al hacer el reemplazo del bloque modificado (el procesador del nodo no esperaba respuesta)
- ☐ Al tener un estado modificado el bloque solicitado por una PtLec o PtLecEx recibida

88. El estado de un bloque de memoria es válido si: 1 punto

Marca solo un óvalo.

- ☐ puede haber copia válida en una o varias cachés
- ☐ hay copia valida en una caché

89. ¿Qué debe cumplirse en un modelo de consistencia secuencial?

1 punto

Selecciona todos los que correspondan.

- ☐ Todas las operaciones de un único procesador (thread) parezcan ejecutarse en el orden descrito por el programa de entrada al procesador (orden del programa))
- ☐ Todas las operaciones de memoria parezcan ser ejecutadas una cada vez (ejecución atómica) -> serialización global
- ☐ Todas las operaciones de un único procesador (thread) deben ejecutarse en el orden descrito por el programa de entrada al procesador (orden del programa))
- ☐ Todas las operaciones de memoria deben ser ejecutadas una cada vez (ejecución atómica) -> serialización global

90. ¿Que requisitos de SC (consistencia secuencial) pueden diferir en los modelos de consistencia relajados?

1 punto

Selecciona todos los que correspondan.

- ☐ Puede alterar el orden del programa al permitir acceso a distintas direcciones (W->R, W->W, R->RW)
- ☐ Puede alterar la atomicidad del programa al permitir al procesador ver el valor escrito por otro antes de que sea visible por el resto de procesadores
- ☐ Permite alterar el paralelismo del programa permitiendo a los procesadores acceder a una misma direccion
- ☐ Permite alterar la ejecución del programa enviandolo al procesador más óptimo(cálculos para GPU, lógica para ALUs...))

91. ¿Que hace la función de cerrojo: unlock(k)?

1 punto

Marca solo un óvalo.

- ☐ Libera a uno de los threads que esperan el acceso a una sección crítica(adquiere el cerrojo)
- ☐ Intenta adquirir el derecho a acceder a una sección crítica (cerrando o adquiriendo el cerrojo k)
- ☐ Pone la hebra que solicita el cerrojo en espera
- ☐ Libera a todas las hebras es que esperan el acceso a la sección crítica(adquieren el cerrojo)

92. ¿Que pasa si varios procesos intentan adquirir un cerrojo a la vez? 1 punto

Marca solo un óvalo.

- ☐ Solo uno debe adquirir el cerrojo, el resto espera
- ☐ Todos adquieren el cerrojo
- ☐ Uno adquiere el cerrojo, el resto ejecuta otra parte del programa
- ☐ Un proceso aleatorio coge el cerrojo y el resto entran en una etapa de espera

93. Si se ejecuta unlock(k) y no hay threads en espera ____ 1 punto

Marca solo un óvalo.

- ☐ La siguiente hebra que ejecute la función lock() adquiere el cerrojo k sin espera
- ☐ La siguiente hebra que ejecuta la funcion lock() entrará en espera igualmente
- ☐ La siguiente hebra que ejecute la función lock() ejecutará la sección critica directamente
- ☐ Las siguientes hebras que ejecuten la función lock() adquieren el cerrojo k sin espera

94. Componentes de un código para sincronización: 1 punto

Selecciona todos los que correspondan.

- ☐ Un método de adquisición. Ej: Load Linked/Store conditional
- ☐ Un método de espera. Ej busy-waiting/bloqueo
- ☐ Un método de liberación mediante cerrojos o barreras
- ☐ Un método de exclusión mutua. Ej: directiva atomic/single
- ☐ Un método de llamada. Ej free-waiting/call

95. Equivalente de abrir un cerrojo (unlock(k)) en OpenMP 1 punto

Marca solo un óvalo.

- ☐ omp_init_lock(&k)
- ☐ omp_set_lock(&k)
- ☐ omp_unset_lock(&k)
- ☐ omp_test_lock(&k)

96. Equivalente de cerrar un cerrojo pero sin bloqueo (unlock(k)) en OpenMP

1 punto

Marca solo un óvalo.

- ☐ omp_unblock_lock(&k)
- ☐ omp_set_lock(&k)
- ☐ omp_unset_lock(&k)
- ☐ omp_test_lock(&k)

97. Características de los cerrojos con etiqueta

1 punto

Selecciona todos los que correspondan.

- ☐ Fijan un orden FIFO en la adquisición del cerrojo
- ☐ Se debe añadir lo necesario para garantizar el acceso en exclusión mutua a contadores
- ☐ Se debe añadir lo necesario para garantizar el orden imprescindible en los accesos a memoria
- ☐ Fijan un orden LIFO en la adquisición del cerrojo

98. Código que implementa la instrucción Test&Set (x)

1 punto

Marca solo un óvalo.

```
temp = x ;  
x = x + a ;  
return (temp)  
/* x compartida,  
a local */
```

☐ Opción 1

```
if (a==x) {  
    temp=x ;  
    x=b; b=temp ; }  
/* x compartida,  
a y b locales */
```

☐ Opción 2

```
temp = x ;  
x = 1 ;  
return (temp) ;  
}  
/* x compartida */
```

☐ Opción 3

99. Código que implementa la instrucción Fetch&Add (x,a)

1 punto

Marca solo un óvalo.

```
temp = x ;  
x = 1 ;  
return (temp) ;  
}  
/* x compartida */
```

☐ Opción 1

```
if (a==x) {  
    temp=x ;  
    x=b; b=temp ; }  
/* x compartida,  
a y b locales */
```

☐ Opción 2

```
temp = x ;  
x = x + a ;  
return (temp)  
/* x compartida,  
a local */
```

☐ Opción 3

100. Código que implementa la instrucción Compare&Swap (a,b,x)

1 punto

Marca solo un óvalo.

```
temp = x ;  
x = 1 ;  
return (temp) ;  
}  
/* x compartida */
```

☐ Opción 1

```
if (a==x) {  
    temp=x ;  
    x=b; b=temp ; }  
/* x compartida,  
  a y b locales */
```

☐ Opción 2

```
temp = x ;  
x = x + a ;  
return (temp)  
}/* x compartida,  
  a local */
```

☐ Opción 3

101. Implementa un cerrojo lock(k) con Test&Set

1 punto

Marca solo un óvalo.

```
1 lock(k) {  
2     while (test&set(k)==0) {}  
3 }  
4 /* k compartida */
```

☐ Opción 1

```
1 lock(k) {  
2     while (test&set(k)==1) {}  
3 }  
4 /* k compartida */  
5
```

☐ Opción 2

```
1 lock(k) {  
2     b=1  
3     do  
4         test&set(k);  
5     while (b==1) {} ;  
6 }  
7 /* k compartida, b local */
```

☐ Opción 3

102. Implementa un cerrojo lock(k) con fetch&or

1 punto

Marca solo un óvalo.

```
1 lock(k) {  
2 |   while (fetch&or(k,1)==1)  
3 | }  
4 /* k compartida */  
5
```

☐ Opción 1

```
1 lock(k) {  
2 |   while (fetch&or(k,1)==0) {} ;  
3 | }  
4 /* k compartida */  
5
```

☐ Opción 2

```
1 lock(k) {  
2 |   b=0  
3 |   do  
4 |     fetch&or(k,b)==1  
5 |     while (b==1);  
6 | }  
7 /* k compartida, b local */  
8
```

☐ Opción 3

103. Implementa un cerrojo lock(k) con compare&swap

1 punto

Marca solo un óvalo.

```

1 lock(k) {
2 |   while (compare&swap(0,1,k)==0) {}
3 | }
4 | /* k compartida */
5 |

```

☐ Opción 1

```

1 lock(k) {
2 |   b=1
3 |   do
4 |       compare&swap(0,b,k)
5 |       while (b==1);
6 |   }
7 | /* k compartida, b local */
8 |

```

☐ Opción 2

```

1 lock(k) {
2 |   while (compare&swap(0,1,k)==1) {};;
3 | }
4 | /* k compartida */
5 |

```

☐ Opción 3

104. [Navegación del cuestionario:]

Marca solo un óvalo.

- ☐ Volver al principio para elegir cuestionario por Tema
- ☐ Tema 1 *Salta a la pregunta 2*
- ☐ Tema 2 *Salta a la pregunta 24*
- ☐ Tema 3 *Salta a la pregunta 63*
- ☐ Tema 4 *Salta a la pregunta 105*
- ☐ Terminar Cuestionario

Tema
4

Son 7 preguntas, hay que hacer tablas y saltos. Gracias a @diegoDeLuis y @thejaimex por su colaboración

107. ¿En qué ciclo se emite la instrucción 5?

1 punto

Marca solo un óvalo.

☐ 7

☐ 8

☐ 6

☐ 5

108. ¿Cuánto tiempo está en la ventana de instrucciones la instrucción 8?

1 punto

Marca solo un óvalo.

☐ 5

☐ 2

☐ 3

☐ 1

109. Teniendo en cuenta que los bits de válido de la instrucción 4 (ADD) son inicialmente 00 (primer 0 para r8 y segundo 0 para r1), ¿Cuáles serán los bits de válido de esta instrucción al terminar el ciclo 5?

1 punto

Marca solo un óvalo.

☐ 00

☐ 01

☐ 10

☐ 11

110. Para implementar el siguiente código, que calcula el producto de una matriz triangular superior por un vector, el compilador ha utilizado saltos condicional negativos (saltos hacia atrás) en lugar de positivos, uno por cada bucle ¿Cuántas entradas de la tabla de saltos (BTC/BTAC) ocupará entonces este trozo de código? 1 punto

```
for(i = 0; i < 8; i++)  
    for(j = i; j < 8; j++)  
        v2[i] += m[i][j] * v1[j];
```

Marca solo un óvalo.

- ☐ 3
- ☐ Dependiendo del valor de j
- ☐ 2
- ☐ Ninguna es correcta
- ☐ Otro: _____

111. Respecto a los saltos de la anterior pregunta. ¿Cuántas veces se ejecuta la instrucción de salto condicional hacia atrás (negativo) que genera el compilador para el bucle interno del código (C) (bucle con índice j) (hay que tener en cuenta que el bucle se ejecuta varias veces al estar dentro de otro bucle)? 1 punto

Marca solo un óvalo.

- ☐ 49
- ☐ 42
- ☐ 24
- ☐ 36

112. [Navegación del cuestionario:]

Marca solo un óvalo.

- ☐ Volver al principio para elegir cuestionario por Tema
- ☐ Tema 1 *Salta a la pregunta 2*
- ☐ Tema 2 *Salta a la pregunta 24*
- ☐ Tema 3 *Salta a la pregunta 63*
- ☐ Tema 4 *Salta a la pregunta 105*
- ☐ Terminar Cuestionario

Este contenido no ha sido creado ni aprobado por Google.

Google Formularios